

ASVA

Technical Requirements Document

Emergency Response & Pre-Hospital Coordination Platform

Version: 2.0

Backend: Firebase

Real-time Database: Cloud Firestore

Medical Media: Cloudinary

Analytics Warehouse: Snowflake

Authentication: Firebase Authentication + Google Sign-In

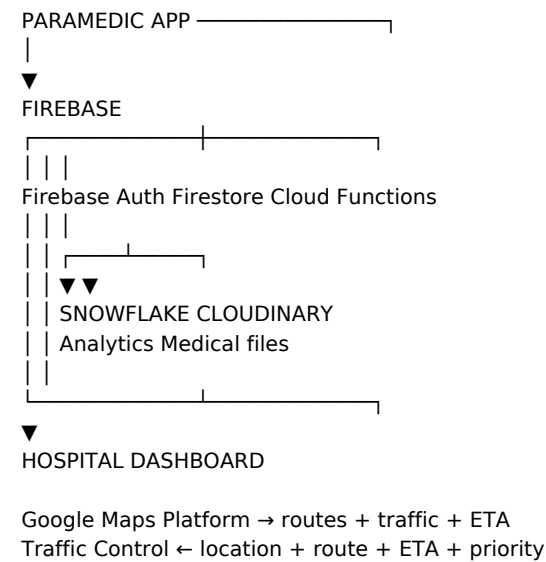
Core principle: Capture less. Understand more. Notify faster.

1. System Objective

ASVA is a real-time emergency coordination system connecting ambulances, traffic-control operations and destination hospitals. The platform reduces pre-hospital coordination time by identifying suitable hospitals, providing route and traffic information, sharing live ambulance movement, capturing important medical parameters through a voice-first workflow, and preparing the destination hospital before arrival.

Core value proposition: Don't make the ambulance wait for the hospital to understand the emergency. Make the hospital ready before the ambulance arrives.

2. High-Level Architecture



3. Technology Stack

Layer	Technology	Responsibility
Frontend	React + TypeScript + Vite	Paramedic and hospital interfaces
UI	Tailwind CSS + shadcn/ui	Responsive interface and components
Backend	Firebase	Application backend
Database	Cloud Firestore	Real-time operational state
Serverless	Firebase Cloud Functions	Business logic and integrations
Authentication	Firebase Authentication	Secure authentication
Auth provider	Google Sign-In	Hospital user login
Medical media	Cloudinary	Actual medical documents and images
Analytics	Snowflake	Historical structured data and analytics
Maps	Google Maps Platform	Map, route, traffic and ETA
Real-time	Firestore listeners	Live dashboard updates
Voice	Speech-to-Text + AI extraction	Voice to structured medical data

4. User Roles

4.1 Paramedic

- Access registered ambulance
- Create emergency case
- Find and select hospital
- View route, traffic and ETA
- Share live location
- Capture vitals
- Use voice-first medical input
- Upload medical reports
- Update treatment and emergency status

4.2 Hospital User

- Sign in with Google through Firebase Authentication
- Access only the authorized hospital
- View incoming ambulances
- View hospital-specific live map
- View ETA, severity and emergency information
- View vitals and treatment
- View authorized report metadata and original documents

Restriction: Google authentication alone does not grant hospital access. The account must be mapped to an authorized hospital.

4.3 Traffic Control

Receives Ambulance ID, Emergency ID where required, current coordinates, route, destination hospital, ETA, severity/priority and timestamp. It must not receive medical reports or unnecessary patient information.

4.4 Citizen

No access to restricted hospital emergency dashboards or medical information.

5. Ambulance and Emergency Identity

Every ambulance is registered in ASVA before an emergency. During an emergency, the paramedic should not waste time entering vehicle information.

Registered ambulance: AMB-1047
Emergency case: EMG-82931

The Ambulance ID and Emergency ID are the primary operational identifiers during transit. Patient name is optional and must never block emergency creation.

6. Emergency Lifecycle

```

CREATED
↓
PATIENT_RECEIVED
↓
HOSPITAL_SELECTED
↓
EN_ROUTE
↓

```

APPROACHING



ARRIVED



COMPLETED

Optional terminal state: CANCELLED

7. Hospital Recommendation

After patient receipt, ASVA presents suitable hospitals. Primary ranking factors are distance and available beds. Emergency capability, operational status and relevant specialization can be applied as additional filters or ranking signals.

Hospital A 4.2 km 12 beds ETA 9 min

Hospital B 5.1 km 20 beds ETA 11 min

Hospital C 7.8 km 8 beds ETA 15 min

The paramedic explicitly selects the destination hospital.

8. Navigation, Traffic and ETA

After hospital selection, ASVA requests a traffic-aware route through Google Maps Platform. The paramedic sees the best route, traffic condition, distance and ETA. If traffic changes materially, route and ETA should be refreshed.

Hospital selected



Google Maps



Route + traffic + ETA



Paramedic navigation



Live updates

9. Live Ambulance Location

The ambulance device periodically obtains GPS coordinates. Current operational location is written to Firestore so hospital dashboards can subscribe to real-time changes.

Ambulance GPS



Firestore



Real-time listener



Hospital dashboard map

Prototype target: location updates approximately every 1–5 seconds, subject to device and network conditions. Snowflake should not be used as the high-frequency live GPS database.

10. Traffic-Control Integration

Firebase Cloud Functions can relay relevant location and route information to a traffic-control integration. Traffic control receives only the minimum data required to coordinate movement.

Firestore



Cloud Function



Traffic-control API / integration



11. In-Transit Medical Monitoring

ASVA must minimize paramedic interaction and capture only clinically important information. Parameters should be prioritized by emergency type.

Emergency type	Priority parameters
General	Emergency type, severity, consciousness, symptoms, key vitals, injury and treatment
Cardiac	Heart rate, blood pressure, SpO ₂ , consciousness, chest pain, ECG
Trauma	Consciousness, BP, HR, SpO ₂ , bleeding, injury location, trauma severity
Respiratory	SpO ₂ , respiratory rate, HR, breathing difficulty, consciousness

12. Voice-to-Structured-Medical Data

Voice entry is a core workflow. ASVA must not simply save a transcript. It must analyze the speech and map recognized clinical values into structured fields.

Example spoken input: "Patient is unconscious, heart rate 120, oxygen saturation 86 percent, blood pressure 90 over 60."

```
{
  "consciousness": "UNCONSCIOUS",
  "heartRate": 120,
  "spo2": 86,
  "bloodPressure": "90/60"
}
```

If the model cannot confidently determine a field, ASVA must flag it for confirmation rather than silently infer a value.

13. Voice Processing Pipeline

```
Voice
↓
Speech-to-Text
↓
Medical entity extraction
↓
Field mapping
↓
Validation
↓
Confidence / ambiguity check
↓
Paramedic confirmation
↓
Firestore
↓
Snowflake
```

Short voice-processing target: under approximately 5 seconds under normal prototype conditions.

14. Firestore Data Model

```
users/{uid}
ambulances/{ambulanceId}
hospitals/{hospitalId}
emergencies/{emergencyId}
```

```
emergencies/{emergencyId}/vitals/{vitalId}
emergencies/{emergencyId}/reports/{reportId}
emergencies/{emergencyId}/locations/{locationId}
```

Firestore contains the real-time subset required by the application. High-frequency location history should be retained selectively rather than creating unnecessary write volume.

15. Example Emergency Document

```
{
  "emergencyId": "EMG-82931",
  "ambulancelId": "AMB-1047",
  "hospitalId": "HOSP-021",
  "emergencyType": "CARDIAC",
  "severity": "CRITICAL",
  "status": "EN_ROUTE",
  "eta": 320,
  "currentLocation": {
    "latitude": 22.5726,
    "longitude": 88.3639
  },
  "createdAt": "...",
  "updatedAt": "..."
}
```

16. Vitals Data

```
{
  "heartRate": 120,
  "bloodPressure": "90/60",
  "spo2": 86,
  "respiratoryRate": 22,
  "temperature": null,
  "consciousness": "UNCONSCIOUS",
  "recordedAt": "..."
}
```

Each vital update must have a timestamp so the patient's in-transit progression can be reconstructed.

17. Cloudinary Medical Media Layer

Cloudinary stores the actual medical documents and images, including ECGs, blood tests, X-rays, CT/MRI images, prescriptions and previous medical reports.

```
ASVA/
├─ emergencies/
│   └─ EMG-82931/
│       └─ reports/
│           ├─ ECG/
│           ├─ BLOOD_TEST/
│           └─ XRAY/
```

18. Cloudinary Upload Flow

```
Paramedic frontend
↓
Request upload authorization
↓
Firebase Cloud Function
↓
Cloudinary upload signature
↓
Direct client upload
```

↓
Clouduinary
↓
Asset ID / reference
↓
Firestore + Snowflake metadata

Large medical files should not unnecessarily pass through Firebase Cloud Functions.

19. Medical Report Metadata

```
{
  "reportId": "REP-5521",
  "emergencyId": "EMG-82931",
  "ambulanceId": "AMB-1047",
  "reportType": "ECG",
  "cloudinaryPublicId": "asva/emergencies/EMG-82931/reports/ECG/001",
  "uploadedBy": "PARAMEDIC",
  "uploadedAt": "..."
}
```

The metadata is stored in the operational layer and synchronized to Snowflake. The actual document remains in Clouduinary.

20. Snowflake Data Warehouse

Snowflake is the structured historical and analytics layer. It should contain emergency history, ambulance and hospital data, vitals history, treatment data, report metadata and selected location history.

Table	Core fields
AMBULANCES	AMBULANCE_ID, VEHICLE_NUMBER, STATUS, REGISTERED_AT
HOSPITALS	HOSPITAL_ID, HOSPITAL_NAME, LATITUDE, LONGITUDE, AVAILABLE_BEDS, EMERGENCY_STATUS, STATUS
EMERGENCY_CASES	EMERGENCY_ID, AMBULANCE_ID, HOSPITAL_ID, EMERGENCY_TYPE, SEVERITY, STATUS, CREATED_AT, UPDATED_AT, ARRIVED_AT
VITALS	VITAL_ID, EMERGENCY_ID, HEART_RATE, BLOOD_PRESSURE, SPO2, RESPIRATORY_RATE, TEMPERATURE, CONSCIOUSNESS, RECORDED_AT
MEDICAL_REPORTS	REPORT_ID, EMERGENCY_ID, AMBULANCE_ID, REPORT_TYPE, CLOUDINARY_PUBLIC_ID, UPLOADED_AT
AMBULANCE_LOCATIONS	LOCATION_ID, EMERGENCY_ID, AMBULANCE_ID, LATITUDE, LONGITUDE, ETA, RECORDED_AT
TREATMENTS	TREATMENT_ID, EMERGENCY_ID, TREATMENT_TYPE, DESCRIPTION, RECORDED_BY, RECORDED_AT
HOSPITAL_USERS	USER_ID, GOOGLE_ID, EMAIL, HOSPITAL_ID, ROLE, CREATED_AT

21. Firebase-to-Snowflake Strategy

Firebase represents the current operational state. Cloud Functions can transform and synchronize relevant events into Snowflake.

LIVE OPERATION
Ambulance GPS → Firestore → Hospital dashboard

HISTORICAL / ANALYTICS
Relevant emergency events → Cloud Functions / pipeline → Snowflake

Do not perform a synchronous Snowflake write for every high-frequency GPS update.

22. Hospital Authentication and Authorization

Hospital user
↓
Google Sign-In
↓
Firebase Authentication
↓
Firebase UID
↓
Authorized hospital lookup
↓
Hospital dashboard

Example authorization profile:

users/UID123
email: doctor@example.com
hospitalId: HOSP-021
role: DOCTOR

Server-side logic and Firestore Security Rules must enforce hospital isolation. Frontend filtering alone is insufficient.

23. Hospital Dashboard

The final major interface is a restricted hospital dashboard.

- Live map showing only ambulances whose selected destination is the authenticated hospital.
- Incoming ambulance list.
- Priority sorting by severity first, ETA second.
- Live vitals, emergency condition and treatment.
- Medical report metadata and controlled document access.

INCOMING

☐ AMB-1047 CRITICAL ETA 5 min
☐ AMB-1082 CRITICAL ETA 11 min
☐ AMB-1031 HIGH ETA 7 min
☐ AMB-1090 MODERATE ETA 4 min

24. Hospital Case Detail

AMB-1047
EMG-82931

CRITICAL
Emergency: CARDIAC
ETA: 05 min

LIVE VITALS
HR: 120 BPM
BP: 90/60
SpO₂: 86%

Condition: Unconscious

Treatment: Oxygen administered

Medical Reports
ECG → View

25. Report Access Flow

Hospital Dashboard
↓
Report metadata
↓
Cloud Function authorization check
↓
Controlled Cloudinary access
↓
Original medical document

Medical reports must not be publicly exposed. Sensitive document access should be controlled and auditable.

26. Real-Time Events

ambulance.location.updated
vitals.updated
emergency.updated
report.uploaded
severity.changed
eta.updated
ambulance.arrived

Firestore listeners should drive hospital dashboard updates without requiring the dashboard to poll continuously.

27. Hospital Arrival

ASVA can use a hospital geofence to transition an ambulance from EN_ROUTE or APPROACHING to ARRIVED. After handoff, the case can transition to COMPLETED.

Ambulance location
↓
Hospital geofence
↓
ARRIVED
↓
Hospital handoff
↓
COMPLETED

28. Firebase Cloud Functions

- createEmergency()
- recommendHospitals()
- selectHospital()
- updateAmbulanceLocation()
- processVoiceInput()
- generateCloudinarySignature()
- createReportMetadata()
- syncEmergencyToSnowflake()
- sendTrafficUpdate()

- calculatePriority()
- detectHospitalArrival()

29. Security Requirements

- Use Firebase Authentication for authenticated hospital access.
- Map authenticated hospital users to an authorized hospitalId and role.
- Enforce hospital data isolation through backend authorization and Firestore Security Rules.
- Keep Cloudinary credentials and Snowflake credentials server-side.
- Do not expose unrestricted medical document URLs.
- Traffic-control integrations receive minimum necessary operational data.
- Patient name remains optional and must not block emergency creation.
- Log sensitive report access where practical.
- Never allow uncertain voice-extracted medical values to be silently accepted.

30. Error Handling

Failure	Required behavior
GPS unavailable	Show location unavailable state, retry and do not present stale coordinates as current.
Voice processing failure	Allow retry and manual entry.
Voice ambiguity	Highlight field and require paramedic confirmation.
Cloudinary upload failure	Allow retry; emergency workflow continues.
Firebase connectivity issue	Show connection status and retry synchronization.
Maps/route failure	Show route error and provide retry/fallback handling.

31. Performance Targets

Operation	Prototype target
Initial hospital dashboard load	< 2 seconds under normal conditions
Live location propagation	< 2 seconds target
GPS update interval	Approximately 1-5 seconds
Short voice processing	< 5 seconds
Normal report upload	< 5 seconds subject to network/file size
Hospital list retrieval	< 2 seconds
Firestore updates	Near real-time

These are hackathon prototype targets, not clinical-grade service-level agreements.

32. Analytics Requirements

Analytics area	Example metrics
Ambulance	Response time, transport time, distance, emergency volume, critical-case percentage
Hospital	Incoming emergency volume, ETA trends, bed availability trends, preparation time

Analytics area	Example metrics
Emergency	Emergency type distribution, severity distribution, peak periods, geographic patterns

33. Suggested Frontend Structure

```
src/
├── components/
│   ├── Map/
│   ├── AmbulanceCard/
│   ├── HospitalCard/
│   ├── VitalCard/
│   ├── ReportCard/
│   ├── VoiceInput/
│   └── EmergencyStatus/
├── pages/
│   ├── Login/
│   ├── Emergency/
│   ├── HospitalSelection/
│   ├── Navigation/
│   ├── MedicalMonitoring/
│   └── HospitalDashboard/
├── services/
│   ├── firebase.ts
│   ├── auth.ts
│   ├── firestore.ts
│   ├── cloudinary.ts
│   ├── maps.ts
│   └── api.ts
├── hooks/
│   ├── useEmergency.ts
│   ├── useLocation.ts
│   ├── useVitals.ts
│   └── useHospitalDashboard.ts
├── types/
└── index.ts
```

34. Environment Configuration

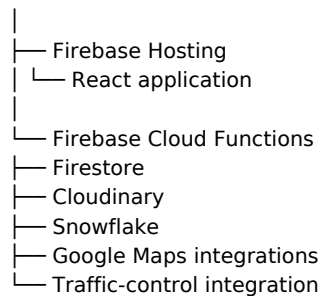
```
Frontend
VITE_FIREBASE_API_KEY
VITE_FIREBASE_AUTH_DOMAIN
VITE_FIREBASE_PROJECT_ID
VITE_FIREBASE_STORAGE_BUCKET
VITE_FIREBASE_MESSAGING_SENDER_ID
VITE_FIREBASE_APP_ID
VITE_GOOGLE_MAPS_API_KEY
VITE_CLOUDINARY_CLOUD_NAME
VITE_CLOUDINARY_UPLOAD_PRESET
```

```
Backend / Cloud Functions
CLOUDINARY_API_KEY
CLOUDINARY_API_SECRET
SNOWFLAKE_ACCOUNT
SNOWFLAKE_USERNAME
SNOWFLAKE_PASSWORD
SNOWFLAKE_DATABASE
SNOWFLAKE_SCHEMA
SNOWFLAKE_WAREHOUSE
GOOGLE_MAPS_API_KEY
```

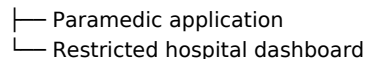
Secrets must remain server-side. Only values explicitly safe for the browser should use VITE_ configuration.

35. Deployment Architecture

INTERNET



Users:



36. One-Day Hackathon MVP

- Registered ambulance and Ambulance ID
- Emergency ID and emergency state
- Hospital recommendation using distance and beds
- Hospital selection
- Traffic-aware route and ETA
- Live ambulance location
- Voice-to-structured medical fields
- Critical vitals
- Cloudinary medical report upload
- Snowflake emergency and report metadata
- Google-authenticated hospital dashboard
- Hospital-specific live map
- Severity + ETA prioritization
- Report metadata leading to the original Cloudinary document

37. Recommended Development Order

Phase	Implementation
1. Firebase	Project, Firestore, Auth, Google provider, Security Rules
2. Ambulance	Registered ambulance, Emergency ID, emergency lifecycle
3. Hospitals	Hospital data, recommendation, distance, beds, selection
4. Maps	Live map, route, traffic, ETA
5. Medical	Vitals, voice extraction, validation and confirmation
6. Cloudinary	Report upload, asset reference, metadata
7. Snowflake	Emergency, vitals and report metadata warehouse
8. Hospital dashboard	Google login, isolation, map, incoming queue, details, report viewer
9. Integration	End-to-end demo and failure testing

38. End-to-End Demo Flow

1. AMB-1047 receives patient.
2. ASVA creates EMG-82931.
3. Hospital list appears with distance and beds.
4. Paramedic selects destination hospital.
5. Route, traffic and ETA appear.
6. Ambulance location begins updating.
7. Paramedic says: "Patient is unconscious, HR 120, SpO2 86, BP 90/60."
8. ASVA converts speech into structured fields.
9. Paramedic confirms the values.
10. ECG is uploaded to Cloudinary.
11. Report metadata is synchronized to Snowflake.
12. Destination hospital receives the incoming ambulance.
13. Hospital dashboard sorts it by severity and ETA.
14. Hospital opens the case and sees vitals and report metadata.
15. Doctor opens the ECG through controlled Cloudinary access.
16. Ambulance moves on the live map.
17. Hospital sees the updated ETA and location.
18. Geofence changes the case to ARRIVED.

39. Final Technology Responsibility Matrix

Technology	ASVA responsibility
React + TypeScript	Paramedic and hospital interfaces
Firebase Authentication	Secure user authentication
Cloud Firestore	Live emergency state, GPS, vitals and dashboard data
Firebase Cloud Functions	Backend logic, authorization, processing and integrations
Cloudinary	Actual medical documents and images
Snowflake	Historical structured emergency data, metadata and analytics
Google Maps Platform	Map, route, traffic and ETA
Voice AI pipeline	Natural speech to structured medical fields

40. Final Architecture Statement

Firebase runs ASVA in real time, Cloudinary manages the actual medical documents, and Snowflake turns ASVA's structured emergency data into a long-term intelligence and analytics layer.

The architecture deliberately separates live operational state, medical media storage and historical analytics. This keeps the ambulance and hospital workflow responsive while giving the hackathon project a credible data architecture for future scale.